Entregável 3 explicado

Rede de distribuição: rede de rotas

Algoritmos e Estruturas de Dados – BCC – 2026.2

Laura da Costa Corte Cordeiro e Ana Helena de Souza Macedo

O que este documento cobre

- Os assuntos cobrados no Entregável 3: grafos, representação esparsa e caminhos mínimos.
- A lógica do arquivo `e3.py`: o grafo dinâmico, a remoção de vértices e o Dijkstra, com os trechos do próprio arquivo.
- Uma legenda com todas as notações, a simulação do Dijkstra no exemplo, complexidades e perguntas da entrevista.
- O heap, o REMOVER e o PAR (E2), o Kadane e a pilha (E1) estão explicados nos documentos anteriores.

Sumário

- 1. Legenda das notações
- 2. O que o Entregável 3 pede
- 3. Assuntos que caem
- 4. Lógica do código: o grafo
- 5. Lógica do código: remoção de vértices
- 6. Lógica do código: DIST com Dijkstra
- 7. Mudanças no main
- 8. Complexidades
- 9. Armadilhas comuns
- 10. Perguntas prováveis da entrevista

1. Legenda das notações

Símbolo	Significado
n	Número de depósitos vivos (vértices do grafo)
m	Número de rotas vivas (arestas do grafo); no setup, m <= 5n
u → v, c	Rota dirigida do depósito u ao depósito v, com custo c (inteiro positivo)
r	Número de rotas que chegam junto com um INSERIR (r <= 20)
g(x)	Grau de x: rotas que saem de x mais rotas que chegam a x
k (em REMOVER)	Número de depósitos removidos por uma chamada de REMOVER
s	Depósito de origem do DIST
dist[v]	Menor custo conhecido de s até v (no código, `distancia`)
−1	Resposta do DIST para um depósito que s não alcança
P, h, T, L_k	Como nos entregáveis anteriores

Nome no código	O que guarda
`rede['saida'][u]`	Dicionário v : custo de todas as rotas que saem de u
`rede['entrada'][v]`	Conjunto dos u que têm rota u → v
`distancia`	Dicionário id : menor custo encontrado a partir de s
`heap`	No DIST, um heap novo com chave = distância (separado da fila de lucros)
`d, u`	Distância e id do vértice tirado do heap
`v, c, nd`	Vizinho, custo da rota u → v e nova distância candidata d + c
`partes`	Pedaços da linha de resposta do DIST

Termo	Significado
Grafo dirigido	Vértices ligados por arestas com direção: rota u → v não implica v → u
Relaxar u → v	Testar se ir até u e depois usar a rota u → v melhora a distância de v
Finalizar u	Ter certeza de que dist[u] é o menor custo possível
Alcançável	Existe algum caminho de s até o vértice seguindo a direção das rotas

2. O que o Entregável 3 pede

Operações exercitadas: **DIST**, com **INSERIR** e **REMOVER** para o grafo mudar (a linha impressa pelo REMOVER também é comparada). As rotas deixam de ser descartadas. O enunciado espera:

- um **grafo dirigido dinâmico**: INSERIR acrescenta rotas e a remoção de um depósito elimina as rotas que o tocam;
- DIST com um algoritmo de caminhos mínimos adequado a custos positivos, **reutilizando a fila de prioridade do E2**;
- a remoção do E2 alcançando o grafo, reutilizando o registro do E1.

Formato do DIST: os custos de s até cada **outro** vivo, em ordem crescente de id, separados por espaço; −1 para quem não é alcançável; linha vazia se s for o único vivo.

3. Assuntos que caem

Tópicos do enunciado: **grafos, representação esparsa e caminhos mínimos**.

3.1 Grafos dirigidos e representações

Representação	Memória	Listar vizinhos de u	Remover um vértice
Matriz de adjacência	$O(n^2)$: 25 milhões de células para n = 5000	O(n)	O(n)
Lista de adjacência (o código)	**O(n + m)**	**O(grau de u)**	**O(grau) com o conjunto de entrada**

O grafo é **esparso** (m é da ordem de n, não de n^2), então a lista de adjacência é a escolha natural.

3.2 Por que guardar também a entrada

Para remover o vértice x, é preciso apagar as rotas que saem de x (fácil: estão em saida[x]) e as que **chegam** a x, que estão espalhadas nos dicionários de outros vértices. Sem o conjunto entrada[x], seria preciso varrer o grafo inteiro. Com ele, o custo é proporcional ao grau de x.

3.3 Caminhos mínimos e o algoritmo de Dijkstra

Dado s, achar o menor custo de s a cada vértice. O Dijkstra mantém distâncias provisórias e, a cada passo, **finaliza** o vértice com a menor distância provisória e **relaxa** as rotas que saem dele.

- **Invariante:** todo vértice já finalizado tem a sua distância correta.
- **Corretude (indução):** quando u sai do heap com a menor distância provisória, qualquer outro caminho até u passaria por um vértice com distância >= a de u e só acrescentaria custo positivo. Logo, dist[u] já é mínima.
- **Por que falha com custo negativo:** uma rota negativa encontrada depois poderia diminuir a distância de um vértice já finalizado.
- Os custos aqui são inteiros de 1 a 10^6, então o Dijkstra é adequado.

Algoritmo	Custo por consulta	Comentário
Bellman-Ford	$O(n \cdot m)$	Aceita custos negativos, mas é lento
Floyd-Warshall	$O(n^3)$	Todos os pares de uma vez; inviável para n = 5000
Dijkstra com heap binário	**O((n + m) log n)**	Só custos não negativos; é o do código

3.4 Alteração de chave ou duplicatas no heap

Quando a distância de v melhora e v já está no heap, há duas opções: **alterar a chave** (o código faz isso, com o heap indexado do E2) ou **inserir uma duplicata** e ignorar a versão velha quando ela sair. A primeira mantém o heap com no máximo n elementos; a segunda é mais simples, mas o heap pode chegar a m elementos. Ambas dão O((n + m) log n).

4. Lógica do código: o grafo

O grafo fica dentro da `rede` em dois campos. **Invariante:** v está em saida[u] se e somente se u está em entrada[v]; só existem chaves de depósitos vivos.

```
def nova_rede():
    return {"lucros": {}, "coords": {}, "fila": novo_heap(), "saida": {}, "entrada": {},
            "total": 0, "par_cache": None}
```

```
def inserir_deposito(rede, ident, x, y, lucro):
    rede["lucros"][ident] = lucro
    rede["coords"][ident] = (x, y)
    heap_inserir(rede["fila"], ident, lucro)
    rede["saida"][ident] = {}
    rede["entrada"][ident] = set()
    rede["total"] += lucro
    rede["par_cache"] = None
```

```
def inserir_rota(rede, u, v, c):
    rede["saida"][u][v] = c
    rede["entrada"][v].add(u)
```

- inserir_deposito cria o vértice **sem rotas** (dicionário e conjunto vazios).
- inserir_rota escreve nos **dois lados**: é isso que mantém o invariante.
- Por isso a ordem importa: o vértice precisa existir antes de receber rotas.

Estado do grafo no exemplo, logo após o setup:

```
saida   = {1: {2: 7, 3: 12}, 2: {3: 2}, 3: {4: 1}, 4: {}}
entrada = {1: set(), 2: {1}, 3: {1, 2}, 4: {3}}
```

5. Lógica do código: remoção de vértices

```
def remover_deposito(rede, ident):
    heap_remover(rede["fila"], ident)
    rede["total"] -= rede["lucros"].pop(ident)
    del rede["coords"][ident]

    saida, entrada = rede["saida"], rede["entrada"]
    for u in entrada.pop(ident):
        del saida[u][ident]
    for v in saida.pop(ident):
        entrada[v].discard(ident)

    rede["par_cache"] = None
```

- Para cada u em entrada[x]: apaga x de saida[u] (as rotas que chegavam a x).
- Para cada v em saida[x]: tira x de entrada[v] (as rotas que saíam de x).
- pop devolve o conteúdo e já apaga a chave de x dos dois dicionários.
- Custo O(log n + g(x)): o heap, mais uma operação O(1) esperado por rota que toca x.

O `remover_abaixo` não mudou: como ele já chamava a função única de remoção, passou a apagar as rotas automaticamente. A complexidade agora inclui os graus: O(k log n + soma dos graus dos removidos).

No exemplo, depois que o REMOVER fecha o depósito 1:

```
saida   = {2: {3: 2}, 3: {4: 1}, 4: {}}
entrada = {2: set(), 3: {2}, 4: {3}}
```

6. Lógica do código: DIST com Dijkstra

```
def heap_extrair_min(heap):
    chave, ident = heap["chaves"][0], heap["ids"][0]
    heap_remover(heap, ident)
    return chave, ident
```

```
def dist(rede, s):
    saida = rede["saida"]
    distancia = {s: 0}
    heap = novo_heap()
    heap_inserir(heap, s, 0)

    while heap["ids"]:
        d, u = heap_extrair_min(heap)
        for v, c in saida[u].items():
            nd = d + c
            if v not in distancia:
                distancia[v] = nd
                heap_inserir(heap, v, nd)
            elif nd < distancia[v]:
                distancia[v] = nd
                heap_alterar(heap, v, nd)

    partes = []
    for ident in sorted(rede["lucros"]):
        if ident != s:
            partes.append(str(distancia.get(ident, -1)))
    return " ".join(partes)
```

Parte do código	O que faz
distancia = {s: 0}	Só s tem distância conhecida; quem não está no dicionário é "infinito"
heap = novo_heap()	Instância **nova** do heap do E2, com chave = distância
heap_extrair_min	Tira o vértice de menor distância: ela já é final
saida[u].items()	Relaxa só as rotas que **saem** de u (respeita a direção)
v not in distancia	Primeira vez que v é alcançado: insere no heap
nd < distancia[v]	Caminho melhor para v, que ainda está no heap: alteração de chave
(nenhum dos dois)	v já foi finalizado ou o caminho não é melhor: nada muda
sorted(rede['lucros'])	Percorre os vivos em ordem crescente de id, pula s e usa −1 para os não alcançados

Por que um vértice já finalizado nunca é alterado: se v saiu do heap, distancia[v] <= d, e como c > 0, nd = d + c > distancia[v].

6.1 Simulação: DIST 1 no exemplo

Passo	Sai do heap	Relaxa	distancia depois	Heap depois
0	—	—	{1: 0}	[1:0]
1	1 (0)	1→2 (7): novo; 1→3 (12): novo	{1:0, 2:7, 3:12}	[2:7, 3:12]
2	2 (7)	2→3: 7 + 2 = 9 < 12, altera	{..., 3: 9}	[3:9]

Passo	Sai do heap	Relaxa	distancia depois	Heap depois
3	3 (9)	3→4 (1): novo, 10	{..., 4: 10}	[4:10]
4	4 (10)	nenhuma rota sai de 4	—	[ ]

Saída, com os vivos 2, 3 e 4 em ordem: **7 9 10**. Depois que o depósito 1 sai e o 5 entra com as rotas 2→5 (2) e 5→4 (4), o DIST 2 dá **2 3 2**: 2→3 = 2, 2→4 = min(3, 2 + 4) = 3 e 2→5 = 2. E, depois que o 2 sai, o DIST 3 dá **1 −1**: o 5 ficou sem caminho.

7. Mudanças no main

```
rede = nova_rede()
for _ in range(n):
    ident, x, y, lucro = (int(t) for t in tokens[pos:pos + 4])
    inserir_deposito(rede, ident, x, y, lucro)
    pos += 4
for _ in range(m):
    u, v, c = int(tokens[pos]), int(tokens[pos + 1]), int(tokens[pos + 2])
    inserir_rota(rede, u, v, c)
    pos += 3
```

```
if op == "INSERIR":
    ident, x, y, lucro, r = (int(t) for t in tokens[pos:pos + 5])
    pos += 5
    inserir_deposito(rede, ident, x, y, lucro)
    for _ in range(r):
        u, v, c = int(tokens[pos]), int(tokens[pos + 1]), int(tokens[pos + 2])
        inserir_rota(rede, u, v, c)
        pos += 3
```

```
elif op == "DIST":
    s = int(tokens[pos])
    pos += 1
    saida.append(dist(rede, s))
```

- **Setup:** todos os depósitos são criados antes de ler as m rotas.
- **INSERIR:** cria o depósito e só depois lê as r rotas, em qualquer direção (o novo id pode ser u ou v).
- **DIST:** lê s e chama dist. Se s for o único vivo, dist devolve texto vazio, que vira uma linha vazia na saída.

8. Complexidades

Operação	Tempo	Justificativa
inserir_rota	O(1) esperado	Uma inserção em dicionário e uma em conjunto
INSERIR	O(log n + r)	Heap mais r rotas de O(1)
Remover um depósito	O(log n + g(x))	Heap mais uma operação por rota que toca x
REMOVER	O(k log n + soma dos graus)	k remoções
DIST	O((n + m) log n)	Cada vivo entra e sai do heap uma vez; cada rota é relaxada uma vez, com no máximo uma alteração de chave; mais O(n log n) para ordenar os ids da saída
Demais	Como no E1 e no E2	LUCRO O(log n), PAR O(n log n), MELHOR e PROSPERIDADE O(1)

Espaço: O(n + m) para o grafo; o DIST usa O(n) a mais para as distâncias e o heap. Com no máximo 200 DIST, o custo total fica folgado.

9. Armadilhas comuns

Erros que derrubam o E3

- Percorrer as rotas na direção errada (usar entrada em vez de saida no Dijkstra).
- Remover o vértice e deixar rotas pendentes nos dicionários dos vizinhos.
- Imprimir o custo até o próprio s ou fora da ordem crescente de id.
- Esquecer o −1 para inalcançáveis ou a linha vazia quando s está sozinho.
- Adicionar as rotas do INSERIR antes de criar o depósito novo.
- Usar a fila de lucros no Dijkstra: misturar as duas instâncias corrompe o REMOVER.
- Continuar pulando as rotas (pos += 3·m ou 5 + 3·r) como no E2.

10. Perguntas prováveis da entrevista

Pergunta	Ideia central da resposta
Por que lista de adjacência e não matriz?	Grafo esparso: matriz teria n^2 células e listar vizinhos custaria O(n).
Para que o conjunto de entrada?	Apagar as rotas que chegam a um vértice removido sem varrer o grafo.
Por que Dijkstra funciona aqui?	Custos positivos: o vértice tirado do heap não pode ser melhorado depois.
Quando o Dijkstra falharia?	Com custos negativos: um vértice finalizado poderia ser melhorado.
Como o Dijkstra reutiliza o heap do E2?	Mesmas funções, outra instância (novo_heap), chave = distância.
Decrease-key ou duplicatas?	O código altera a chave (heap indexado); duplicatas seriam mais simples, mas o heap cresce até m.
Qual a complexidade do DIST?	O((n + m) log n), mais O(n log n) para ordenar a saída.
O que muda se houver 10^6 depósitos?	Lista de adjacência e Dijkstra ainda escalam; matriz e Floyd-Warshall não; o PAR em O(n log n) também aguenta.